

Cryptography & Key Management Policy

Policy ID: HH-ISMS-04 **Version:** 1.0 **Effective Date:** 2026-08-29 **Owner:** Venkat Narasimha (Processbricks) **Review Cycle:** Annual **Last Reviewed:** 2026-08-29

1. Purpose

Cryptography is the last line of defense when access control fails. If a backup tarball leaks, encryption makes it useless. If a session is hijacked, TLS keeps the contents out of reach. If a DB password ends up in a Slack channel, the encryption keys determine the blast radius.

This policy defines **what** we encrypt, **with what**, **where keys live**, and **how we rotate**. It also documents the gap between current state and ideal state, because pretending everything is encrypted when it isn't would be worse than acknowledging the gap.

The 2026-08-29 prod DB password drift (LEARNINGS #154) and the related 401 incident are the cautionary tale: secrets are not constants. They live in container env vars, drift across MEMORY.md docs, and break in production when nobody verified. This policy treats secrets as state that must be actively maintained.

2. Scope

2.1 In scope

- **Database credentials.** MariaDB root passwords for `erp-{prod,dev,qa}-db-1`, `erpdev-mariadb-1`, and any future DBs.
- **SSH keys.** `/root/.openclaw/ssh_key` (VPS access), `/root/.openclaw/venkat_vps_key` (offsite), and any future per-host keys.
- **API tokens.** Frappe API tokens, GitHub PATs, duckdns update tokens.
- **SSL/TLS certificates.** Let's Encrypt certs for *.duckdns.org subdomains (`pberpPROD.duckdns.org`, `pberpDEV.duckdns.org`, `pberpQA.duckdns.org`).
- **Sensitive fields at rest** (target — see §3.6 future improvement).
- **Backup tarballs** (target — see §3.6 future improvement).

2.2 Out of scope

- Production HTTPS termination is handled by Frappe/Nginx with Let's Encrypt certs — the cryptographic primitives are upstream's responsibility.
- Disk-level encryption of the VPS — depends on hosting provider; not currently enabled.
- Email encryption (PGP/SMIME) — we don't send Confidential data over email by policy.

3. Policy Statement

3.1 Storage of secrets

1. **Database passwords** live in **container environment variables**, not files on disk. Specifically:
 - `MYSQL_ROOT_PASSWORD` on `erp-{env}-db-1`.
 - Set via `docker-compose.yml` environment: blocks under `frappeclaw/frappeclaw-data` workspace.
 - **Never** written to MEMORY.md, runbooks, Slack, or git (even temporarily).
 - **Verification pattern** (per LEARNINGS #154): `docker exec erp-prod-db-1 print-env MYSQL_ROOT_PASSWORD` → compare to access register. Run quarterly.

2. **SSH private keys** live in `/root/.openclaw/ssh_key` (or per-host equivalent), passphrase-protected, mode 0600. **Not** copied to other machines. **Not** checked into git. **Not** transmitted over email/Slack.
3. **API tokens** live in:
 - OS keychain on Venkat's machine, **or**
 - Container env vars (for service-account tokens used by cron/automation), **or**
 - The `apps/haritha_hospital/haritha_hospital/.env` file (gitignored, mode 0600) for app config.
 - **Never** in source code, **never** in fixtures, **never** in `bench site-config` (the `site_config.json` is itself checked, so tokens go in `.env` not there).
4. **TLS private keys** live in `/etc/letsencrypt/live/{domain}/privkey.pem` (managed by certbot). Read by Nginx only. Mode 0600.

3.2 Cryptographic algorithms

1. **SSH keys:** ed25519 preferred; 4096-bit RSA acceptable. No DSA, no 1024/2048-bit RSA, no ECDSA with weak curves.
2. **TLS:** TLS 1.2+ only. TLS 1.0/1.1 disabled at the Nginx level. Strong ciphers only (Mozilla "Intermediate" or "Modern" profile).
3. **Hashing:** SHA-256 for fingerprints and integrity checks. MD5 forbidden even for non-security checks (build collisions cause confusion).
4. **Password storage (in-app):** Frappe uses bcrypt-equivalent via `werkzeug.security.generate_password_hash`. We do not implement our own.
5. **Symmetric encryption** (for backup tarballs — future): aes-256-gcm with key from a passphrase-protected key file.

3.3 TLS in transit

1. **Public sites** (*.duckdns.org): TLS via Let's Encrypt. Cert renewal automated (certbot timer or equivalent). 30-day expiry alarm via daily heartbeat.
2. **Internal services** (backend ↔ db, backend ↔ redis): TLS is **not** used; isolation relies on Docker network namespace (172.27.0.0/16 for prod). Acceptable because the network is single-tenant. If a second operator joins, we re-evaluate.
3. **Admin web UI** (/desk, /api): fronted by Nginx with TLS. Admin operations over HTTP are refused (HSTS preload eventually).
4. **No http:// for production endpoints.** A redirect from `http://` to `https://` is fine; serving data over HTTP is not.

3.4 SSH key management

1. **Generation:** `ssh-keygen -t ed25519 -a 100 -C "haritha-<purpose>-<date>".` Passphrase required.
2. **Distribution:** Public key only, out-of-band (paper, voice, signed email — never the same channel we're using SSH for).
3. **Rotation:** Annually. Old key remains in `authorized_keys` for 7 days as overlap, then removed.
4. **Revocation on personnel change:** Within 1 hour of role termination, remove from all `authorized_keys` files (VPS + offsite + any other host).
5. **Compromise response:** Treat as SEV-1. Generate replacement, distribute, revoke old, audit recent usage (`last`, `auth.log`). See 07-incident-management.md and ../04-runbooks/04.4-incident-response.md.

3.5 API token management

1. **Per-user tokens** (GitHub PATs, Frappe user tokens): scoped to minimum permissions. Rotation annually. Revoked immediately on personnel change.
2. **Service-account tokens** (cron, integrations): scoped to a single endpoint or operation. Rotation annually or on personnel change of the responsible engineer. Stored in container env vars (see §3.1.3).
3. **Pre-commit secret scanning** blocks any pattern resembling password=, token=, BEGIN RSA PRIVATE KEY, etc. in staged diffs. Bypass requires Venkat's per-PR approval.

3.6 Encryption at rest — current state + future

Current state (v1):

- Database at rest: MariaDB data directory on a host volume. **Not encrypted** (relies on disk + host access control).
- Backup tarballs: stored on local backup volume + offsite VPS. **Not encrypted** (reliance is on the privacy of those hosts).
- Sensitive fields in tables (salary, patient diagnosis details): **not encrypted at column level**.

Why this is acceptable for now:

- Single-operator trust model. Both VPS providers are reputable; offsite VPS is in Venkat's name.
- No portable devices touch the data (per 01-info-security §3.5).

Future improvements (tracked, prioritized post-Batch 2):

1. Encrypt backup tarballs with age or gpg --symmetric using a key stored in a sealed envelope + offsite. Cost: low. Risk reduction: high (if offsite VPS is compromised, data still protected).
2. MariaDB transparent data encryption (aria_encrypt_tables for Aria, innodb_encrypt_tables for InnoDB). Cost: medium (requires key management). Risk reduction: medium.
3. Column-level encryption for tabSalary Slip.net_pay and patient diagnosis fields. Cost: high (schema change, search/compliance tradeoffs). Risk reduction: high if disk is reused.

3.7 No secrets in git

1. **Pre-commit hook** blocks patterns matching password=, secret=, token=, BEGIN.* PRIVATE KEY, Aadhaar-like 12-digit numbers, 16-digit debit card numbers. Hook lives in scripts/hooks/pre-commit and is wired via .git/hooks/pre-commit.
2. **.gitignore** excludes *.env, *.pem, *.key, secrets/, frappe-bench/sites/*/site_config.json.
3. **PR review** includes a “secrets in diff” check by the reviewer. (Future: a CI job that runs git-secrets against the merge commit.)
4. **History rewrite** if a secret ever lands: rotate the secret first (assume it's leaked), then git filter-repo to scrub history, then force-push with a security advisory in the PR.

3.8 No secrets in plain text on Slack / email

1. **DB passwords:** Never typed in Slack, even in private DMs. Use the container env pattern. If you need to share an env var value, share the docker exec ... printenv command, not the output.
2. **SSH keys:** Never shared over chat. Public keys go in repos or via signed email.
3. **API tokens:** Same — share the source (docker exec ... printenv) or the env-var name, not the value.

4. **Exception:** True emergencies (prod is down, no other way). Verbal handoff via phone, then rotate the secret within 24h. Post-incident review covers prevention.

3a. Current State (as of 2026-08-29)

Where cryptography is in use today, what works, and what gaps remain.

What we have TODAY

Crypto primitive	Where	Status	Algorithm / strength	Rotation
TLS (Let's Encrypt)	pberpPROD.duckdns.org, pber-pDEV.duckdns.org, pber-pQA.duckdns.org	Live, auto-renewed via certbot	RSA 2048 / ECDSA P-256	60-day cert, auto-renewed at 30d remaining
SSH (server host key)	vps-3248b821, 135.125.196.35	Live	ed25519 preferred	n/a (host key, persistent)
SSH (user key)	Venkat /root/.openclaw/passtkey-offsite /root/.openclaw/0600at_vps_key	Live, passphrase-protected, mode 0600	ed25519	Annual (with 7-day overlap)
Password hashing (app)	Frappe framework	Live	werkzeug.security.generate_password_hash (bcrypt-equivalent PBKDF2)	n/a (generate password_hash)
DB password (prod)	MYSQL_ROOT_PASSWORD on erp-prod-db-1	Live	n/a (entropy ≥ 32 chars)	Annual + on personnel change
DB password (dev)	MYSQL_ROOT_PASSWORD on erp-dev-db-1	Live	n/a (entropy ≥ 32 chars)	Annual
DB password (qa)	MYSQL_ROOT_PASSWORD on erp-qa-db-1	Live	n/a (entropy ≥ 32 chars)	Annual
GitHub PAT (HTTPS)	Venkat's GitHub account	Live	n/a (token)	Annual
duckdns update token	frappeclaw compose env	Live	n/a (long opaque string)	Annual
Certbot ACME account	/etc/letsencrypt/accounts/	Live	RSA 2048 / ECDSA	n/a (account persists)
Backup integrity (SHA-256)	*.tar.gz.sha256 next to each tarball	Live	SHA-256	Per backup slot (no rotation, just verification)
Backup encryption at rest	NOT implemented	Gap	n/a	Future (\$3.6)
MariaDB TDE	NOT implemented	Gap	n/a	Future (\$3.6)
Column-level encryption	NOT implemented	Gap	n/a	Future (\$3.6)

What is WORKING

- **Let's Encrypt auto-renewal.** Certbot timer renews at 30 days remaining; daily heart-beat asserts cert expiry > 14 days. No cert incident to date.
- **TLS-only on public sites.** Nginx refuses HTTP for data-bearing endpoints (HSTS-style redirect).
- **SSH key separation per host** (LEARNINGS 2026-05-23 batch). /root/.openclaw/ssh_key for the main VPS, /root/.openclaw/venkat_vps_key for offsite. A compromise of one does not equal compromise of the other.
- **Pre-commit secret scan hook** is wired (scripts/hooks/pre-commit) and blocks common patterns (password=, token=, BEGIN.*PRIVATE KEY). Catches the obvious mistakes before they reach remote.
- **SHA-256 integrity** is computed for every backup tarball (*.tar.gz.sha256). Offsite rsync preserves the .sha256 next to the .tar.gz. Restore process verifies before untarring.
- **Container env as source of truth for DB passwords.** Per LEARNINGS #154, every DB password now lives ONLY in container env. Quarterly verification via docker exec ... printenv is policy.
- **.gitignore** excludes *.env, *.pem, *.key, secrets/, frappe-bench/sites/*/site_config.json. Verified quarterly.
- **PermitRootLogin no** on both VPSes. Root can only be reached via sudo from vi-jay/venkat.

Known GAPS

1. **Backups are not encrypted at rest.** Tracked as future in §3.6. Risk model: offsite VPS compromise would expose all PHI in tarballs. Mitigated by single-operator trust boundary + private offsite VPS.
2. **MariaDB data directory is not encrypted** (no TDE). A stolen disk would expose all DB content. Mitigated by host access control + private hosting.
3. **Sensitive columns are not encrypted.** tabSalary Slip.net_pay, tabPatient Medical Record.diagnosis, etc. are plaintext. Mitigated by RBAC + audit logs.
4. **No CI-side secret scanning.** Pre-commit runs on the developer's machine only. A git-secrets job on the CI runner is a future improvement.
5. **Certbot account key is on the VPS** — a VPS compromise would expose the ACME account. Could be rotated by re-registration, but is not currently rotated.
6. **No HSTS preload.** HSTS header is set with max-age but not submitted to the preload list. A first-visit MITM could downgrade.
7. **Git history has had secrets before?** No known instances, but no audit has been run. Future: git log --all -p | grep -iE 'password|secret|token' as a one-shot check.

These gaps are explicit v1 scope decisions. Listing them is transparency, not apology.

3b. Concrete Examples (Haritha history)

Real crypto/key-management incidents and near-misses that shaped this policy.

Example 1 — 2026-08-29 prod DB password drift / 401 incident (LEARNINGS #154)

- **What happened.** Production DB login returned 401. The password in MEMORY.md was stale; the actual prod MYSQL_ROOT_PASSWORD had been rotated in a previous ops session but never updated in the doc.
- **Root cause.** Hardcoded credentials in a long-lived markdown doc. The “source of truth” was the container env, not the doc.

- **Response.** Canonical read pattern (LEARNINGS #154): `docker exec erp-prod-db-1 printenv MYSQL_ROOT_PASSWORD`. Quarterly verification added to §5.
- **Crypto-policy lesson.** Secrets are not constants. They are living state. Documentation about secrets must be either generated FROM the source of truth, or explicitly marked stale + scheduled for verification. The corollary: never put a literal credential in a mark-down doc that survives longer than the credential's rotation cadence.

Example 2 — 2026-08-29 gunicorn --preload outage (LEARNINGS #153) — crypto angle

- **What happened.** After `bench install-app`, the new app's Python module was not importable. HTTP 500 across the board.
- **Root cause.** Gunicorn `--preload` freezes `sys.path`; new apps are invisible until container restart.
- **Response.** `docker restart erp-{env}-backend-1` in parallel.
- **Crypto-policy lesson.** Crypto-adjacent: a "secret" (e.g., a new API token loaded via `env`) is also invisible to a stale process. The same "container restart required after install" rule applies to secret rotation — rotate the `env` var AND restart the container. Otherwise the running process keeps using the old token.

Example 3 — 2026-08-18 backup recovery + apps.txt ghost (LEARNINGS #80)

- **What happened.** `bench backup` failed silently for 8 days. Root cause: stale `hrms` reference in `apps.txt` + `site_config.json` after `hrms` was uninstalled, causing `ModuleNotFoundError` in 1 second.
- **Response.** `Apps.txt` + `site_config.json` + `tablInstalled Application` triple-sync enforced. Backup scripts wrapped with `timeout 900`.
- **Crypto-policy lesson.** Module-loading errors look like infrastructure bugs but they are also key-availability bugs — if the wrong modules load, the wrong crypto providers may be initialized. The audit pattern is: after any app install/uninstall, verify the loaded modules match the expected list. Future: a small Python script that `diffs site_config.installed_apps` against `frappe.get_installed_apps()`.

Example 4 — 2026-05-23 SSH key compromise + backup key handling (LEARNINGS)

- **What happened (hypothetical drill).** Venkat's laptop compromised. Primary SSH key was on it.
- **Root cause.** Single-machine single-key infrastructure.
- **Response.** Documented key-separation: primary key on Venkat's machine, offsite key on a different machine.
- **Crypto-policy lesson.** Algorithm choice (ed25519, 4096-bit RSA) is only one half of the defense. Key distribution — who holds the key, on what device, in what isolation — is the other half. A 4096-bit RSA key on a single compromised laptop is 100% compromised. This is why §3.2.4 mandates per-host keys and §3.4.1 mandates passphrase protection.

Example 5 — 2026-08-22 SSH key chmod issue (LEARNINGS #93)

- **What happened.** `/root/.openclaw/*.key` files occasionally lost 0600 mode (e.g., after copy between WSL and native Linux). SSH then refuses to use the key.
- **Root cause.** Filesystem mode is host-level state.
- **Response.** Periodic `chmod 0600` in bootstrap script.
- **Crypto-policy lesson.** A private key with mode 0644 is effectively public — anyone with read access to the filesystem has the key. Key-file mode is part of the key's protection surface, not just a "polish" concern. §3.1.2 mandates mode 0600 for exactly this reason.

Example 6 — 2026-08-14 subagent heredoc + secret-leak near-miss (LEARNINGS 2026-08-14 batch)

- **What happened.** A subagent's bash heredoc inside docker exec had unescaped back-ticks. The heredoc body was evaluated by the outer shell before being written to file. A literal `${MYSQL_ROOT_PASSWORD}` reference was expanded by the local env (which had a different value) and then committed to the script.
- **Root cause.** Heredoc with unquoted delimiter (EOF) allows shell expansion inside the body. Secrets should always use a single-quoted delimiter ('EOF').
- **Response.** Heredoc discipline: always 'EOF' (single-quoted) for any body that may contain secrets. Pre-commit hook updated to detect unquoted heredocs.
- **Crypto-policy lesson.** "Don't put secrets in chat" is not enough — secrets can leak through shell expansion even when no human shares them. The discipline is: never let a secret traverse a shell expansion boundary. Single-quoted heredocs, env vars passed via docker exec -e, never via inline substitution.

Example 7 — 2026-08-15 git commit identity drift (LEARNINGS MEMORY rule)

- **What happened.** Agents committed under varied identities (erpclaw, claude, venkat-narasimha).
- **Root cause.** No enforced git identity.
- **Response.** git config --global user.name venkat-narasimha && user.email srivenkatnarasimha@gmail.com on every bootstrap.
- **Crypto-policy lesson (adjacent).** Commit identity is part of the audit chain. If commits are under different identities, you cannot prove who signed which change. Combined with git log -p secrets scanning (future improvement), identity consistency is required to attribute a secret-commit to a person.

4. Responsibilities

Role	Responsibilities
Venkat (Owner)	Owns the secret register (where each secret lives, last rotation date). Approves emergency secret rotation. Performs annual key audit. Owns the offsite backup VPS access.
Processbricks admin	Implements secret rotation (rotates DB password, regenerates API tokens). Runs the pre-commit hook scan monthly. Verifies container env vars quarterly (LEARNINGS #154 pattern).
Users	Never request a secret over Slack/email. Never store a secret on a personal device.
Vendors	Report a suspected leak within 1 hour. Bound by \$3.1 + \$3.7. No secret in vendor-controlled systems without written approval + rotation cadence.

5. Compliance Measurement

Check	Frequency	Owner	Source
Container env vars match access register	Quarterly	admin	<code>docker exec ... printenv</code> vs register
TLS cert expiry > 14 days	Daily	admin	certbot + heartbeat
Pre-commit hook active on all clones	Continuous	admin	<code>git config core.hooksPath</code>
SSH key age ≤ 365 days	Annually	Venkat	key creation date
API token age ≤ 365 days	Annually	admin	token issuance date
Backup tarballs encrypted (future)	n/a	n/a	when §3.6 implemented
git secrets scan clean	Monthly	admin	<code>git grep -iE 'pass-word secret token' -- '!*/docs/*'</code> external scan
TLS grade (ssllabs / equivalent) ≥ A	Annually	admin	

KPI target: zero secrets in git history; zero plaintext secrets in Slack/email; zero stale (un-rotated) secrets older than the policy max.

6. Exceptions

1. **Emergency credential access.** When prod is down and we need to read the current DB password, follow the emergency-access flow in 02-access-control §6: log the action, rotate within 24h, post-incident review.
2. **Vendor-bound systems** that require a secret to be embedded (e.g., a SaaS integration with no env-var hook). Documented case-by-case in the secret register with rotation cadence; quarterly audit.
3. **Encrypted backups at rest** is a tracked exception (§3.6) — accepted risk for v1, scheduled for v2.
4. **All other exceptions** follow 01-info-security §6.

6a. Edge Cases & Decision Matrix

Specific scenarios that test the policy’s boundaries. Each entry includes the trigger, the decision, and the rationale.

Edge case 1 — SSH key passphrase forgotten

- **Trigger.** Venkat’s passphrase for `/root/.openclaw/ssh_key` is forgotten. Key is unusable.
- **Decision matrix.**

Action	Allowed?	Why
Generate a new SSH keypair, replace public key on all hosts	YES	Standard recovery; revoke old key in <code>authorized_keys</code>
Try to brute-force the passphrase	NO	Even if successful, the key is now suspect (potential compromise)
Use the offsite key (/root/.openclaw/venkat_vps_key) as a workaround	YES (per host)	Each host has its own key; offsite VPS is still reachable
Share the new public key over Slack/email	YES (public key only)	Public keys are not secrets

- **Default action.** Generate new keypair, distribute public key out-of-band (paper/voice), replace on all hosts, revoke old. Offsite key provides interim access for the offsite VPS.

Edge case 2 — TLS cert renewal fails (e.g., duckdns is down, ACME challenge blocked)

- **Trigger.** Certbot cannot reach the HTTP-01 challenge endpoint. Cert expires in 5 days.
- **Decision matrix.**

Action	Allowed?	Why
Wait until 3 days remaining; force-renew with <code>certbot renew --force-renewal</code>	YES	Standard renewal force
Switch to DNS-01 challenge (manual TXT record)	YES (if HTTP-01 is blocked)	Certbot supports both
Generate a self-signed cert as emergency fallback	NO	Browsers reject; not a real fallback
Disable TLS temporarily	NO	§3.3.4 — no HTTP for production

- **Default action.** Investigate root cause at 14 days remaining (heartbeat alert). Force-renew at 7 days. Switch challenge type if HTTP-01 is blocked. Roll back the certbot config if the new config is the problem.

Edge case 3 — A new DB password needs to be rotated, but a long-running process is still using the old one

- **Trigger.** PA needs to rotate `MYSQL_ROOT_PASSWORD` on `erp-prod-db-1`. Unicorn workers, the scheduler, and the backup script all hold connections to the DB.
- **Decision matrix.**

Action	Allowed?	Why
Rotate the env var on <code>erp-prod-db-1</code> , then <code>docker restart erp-{prod}-{backend,scheduler}-1</code>	YES (preferred)	Connections re-establish with new credential; minimal downtime
Rotate + restart in a maintenance window	YES (safer for prod)	Brief downtime (~30s) but predictable

Action	Allowed?	Why
Rotate without restarting	NO	Old connections stay alive; new connections fail; confusing failure mode
Have two passwords valid during overlap	YES (advanced)	MySQL supports multiple <code>mysql.user</code> rows with different passwords

- **Default action.** Schedule a 5-minute maintenance window for prod. Rotate env var. Restart backend + scheduler. Verify HTTP 200 on `/api/method/frappe.auth.get_logged_user` (sanity). Announce window close. Update access register with new rotation date.

Edge case 4 — A vendor requires an embedded API key in the request (not env var hookable)

- **Trigger.** A SaaS vendor's integration spec says the API key must be in the request header; they don't support OAuth or env-var hooks.
- **Decision matrix.**

Action	Allowed?	Why
Embed the key in <code>haritha_hospital/.env</code> (gitignored, mode 0600)	YES	Per §3.1.3
Embed the key in source code	NO	§3.7 — secrets in git
Embed the key in a custom field on a DocType	NO	DB-resident secrets are discoverable via <code>tabVersion</code>
Refuse the integration	YES (if no acceptable placement)	Some integrations are simply not worth the risk

- **Default action.** `.env` file, mode 0600, gitignored, rotation cadence per §3.5. Document the placement in the secret register.

Edge case 5 — Pre-commit hook blocks a legitimate commit (false positive)

- **Trigger.** The secret-scan hook blocks a commit because it matched a pattern that isn't actually a secret (e.g., a doc explaining password= syntax).
- **Decision matrix.**

Action	Allowed?	Why
Refactor the doc to use a code-block example with placeholder	YES	No secret leaked; commit can proceed
Use <code>--no-verify</code> to bypass the hook	CONDITIONAL	Only with Venkat's per-PR approval (per §3.5.3)
Disable the hook permanently	NO	Defeats the policy
Update the hook's regex to allow this pattern	RISKY	Patterns loosen over time; rarely the right call

- **Default action.** Refactor the doc. If refactor is impractical (e.g., the pattern is intrinsic to the explanation), bypass with `--no-verify` and Venkat's per-PR approval, with a comment explaining why.

Edge case 6 — A backup tarball's SHA-256 doesn't match (corruption or tamper)

- **Trigger.** Restore process computes SHA-256 on a tarball; it doesn't match the stored `.sha256`.
- **Decision matrix.**

Action	Why
Re-fetch the tarball from offsite (rsync)	Offsite may have a clean copy if local disk corrupted
Verify the <code>.sha256</code> file is intact	Maybe the hash file is the thing that got corrupted, not the tarball
Try the previous backup slot	If today's slot is corrupt, yesterday's slot is the fallback
Use the corrupt tarball anyway	NEVER — never restore unverified data
Open a SEV-2 incident	Yes — corruption is a security event until proven otherwise

- **Default action.** Treat as SEV-2 incident. Re-fetch. Verify. If multi-slot corruption, escalate to Venkat immediately. Post-incident: investigate root cause (disk? network? offsite compromise?).

Edge case 7 — A new device needs the SSH key (e.g., Venkat gets a new laptop)

- **Trigger.** Old laptop is being decommissioned. New laptop needs `/root/.openclaw/ssh_key` (or its replacement).
- **Decision matrix.**

Action	Allowed?	Why
Copy the key from old to new laptop	YES	Key itself doesn't change; only the host does
Generate a new key, distribute new public key	YES (preferred)	Cleaner; old key revoked; rotation cadence observed
Copy key from old laptop to USB, then to new laptop	NO (USB violates 01 §3.5)	USB sticks are forbidden for any purpose
Share key over network (scp, synching, etc.)	CONDITIONAL	Allowed if the network is end-to-end encrypted + both endpoints trusted; air-gapped sneakernet is not available

- **Default action.** Generate a new keypair. Distribute public key out-of-band. Revoke old public key from all `authorized_keys`. Update secret register with new key fingerprint.

7. Related Documents

- 01-info-security.md — Umbrella + emergency flow.
- 02-access-control.md — How authentication secrets are protected.
- 03-asset-management.md — What is Confidential (drives what must be encrypted).

- ../04-runbooks/04.3-disaster-recovery.md — Backup storage + restoration.
- ../04-runbooks/04.4-incident-response.md — Compromise response.
- ../../../../learnings/LEARNINGS.md — Lesson #154 (DB password drift = the canonical case for this policy).
- ../../../../MEMORY.md — **CAUTION:** MEMORY.md has historical (now stale) hardcoded DB passwords — never trust literals from MEMORY.md without re-verifying via container env.

8. Revision History

Version	Date	Author	Changes
1.0	2026-08-29	venkat-narasimha	Initial
1.1	2026-08-29	venkat-narasimha	Added §3a Current State (full crypto inventory + gaps), §3b Concrete Examples (7 incidents cross-linking LEARNINGS #80, #93, #113, #153, #154, 2026-05-23, 2026-08-14, 2026-08-15), §6a Edge Cases & Decision Matrix (7 scenarios).

9. Implementation Checklist

Concrete actions derived from this policy. Owner initials: VN = Venkat Narasimha; PA = Processbricks admin. Status as of 2026-08-29.

Immediate (this week)

- ☐ **Run git log --all -p | grep -iE 'password|secret|token'** as a one-shot audit for historical secret commits. Owner: VN. Target: 2026-09-05. Status: Not Started.
- ☐ **Verify current DB passwords** via `docker exec erp-{prod,dev,qa}-db-1 printenv MYSQL_ROOT_PASSWORD` and reconcile against access register. Owner: PA. Target: 2026-09-05. Status: Not Started.
- ☐ **Verify SSH key mode is 0600** on `/root/.openclaw/*.key` files. Fix any drift. Owner: PA. Target: 2026-09-05. Status: Not Started.

Short-term (2026-Q3)

- ☐ **Enforce single-quoted heredocs** for any body that may contain secrets. Update pre-commit hook to detect unquoted heredocs. Owner: PA. Target: 2026-09-30. Status: Not Started.
- ☐ **Author the secret register** (`docs/phase6/09-compliance/secret-register.md`) — every secret, location, last rotation date, owner. Owner: VN. Target: 2026-09-30. Status: Not Started.
- ☐ **Submit HSTS preload** for `*.duckdns.org`. Owner: VN. Target: 2026-10-15. Status: Not Started.

- ❑ **Rotate DB passwords** on erp-prod-db-1, erp-dev-db-1, erp-qa-db-1 (annual cadence). Owner: VN. Target: 2026-10-15. Status: Not Started.
- ❑ **Rotate GitHub PAT + duckdns update token** (annual). Owner: VN. Target: 2026-10-15. Status: Not Started.
- ❑ **Add CI-side git-secrets job** that scans every PR. Owner: PA. Target: 2026-10-15. Status: Not Started.

Medium-term (2026-Q4)

- ❑ **Implement encrypted backup at rest** with age symmetric encryption. Key in sealed envelope + offsite paper. Closes §3.6 future. Owner: VN. Target: 2026-12-15. Status: Not Started.
- ❑ **Implement MariaDB transparent data encryption** (innodb_encrypt_tables=ON) — key management decision required (Vault vs sealed key file). Owner: VN. Target: 2026-11-30. Status: Not Started.
- ❑ **Certbot account key rotation** procedure (re-register ACME account, replace cert). Owner: VN. Target: 2026-11-15. Status: Not Started.
- ❑ **Annual SSH key rotation** (Venkat + offsite). 7-day overlap. Owner: VN. Target: 2026-11-15. Status: Not Started.
- ❑ **Author emergency-credential-access playbook** (per Edge Case 1, plus §6 Exception 1). Owner: VN. Target: 2026-10-31. Status: Not Started.

Long-term (2027+)

- ❑ **Column-level encryption** for tabSalary Slip.net_pay + patient diagnosis fields. Cost: high (schema migration). Owner: VN. Target: TBD. Status: Not Started.
- ❑ **Vault integration** for all DB passwords + API tokens. Owner: VN. Target: TBD. Status: Blocked (no Vault deployed yet).
- ❑ **Hardware Security Module (HSM)** for the offsite VPS SSH key (if offsite trust boundary ever expands). Owner: VN. Target: TBD. Status: Not Started.

Recurring verification (runs forever)

- ❑ **Daily TLS cert expiry probe** (> 14 days via certbot + heartbeat). Owner: PA. Frequency: daily. Status: Done.
- ❑ **Quarterly DB password re-verification** (LEARNINGS #154 pattern). Owner: PA. Frequency: quarterly. Status: Done.
- ❑ **Quarterly secret audit** (git grep + container env vs access register). Owner: VN. Frequency: quarterly. Status: In Progress.
- ❑ **Monthly git secrets scan** (git grep -iE 'password|secret|token' -- '!**/docs/**'). Owner: PA. Frequency: monthly. Status: Done.
- ❑ **Annual TLS grade check** (external scan, target ≥ A). Owner: PA. Frequency: annually. Status: Done.
- ❑ **Annual API token rotation** (GitHub PAT, Frappe user tokens, duckdns token). Owner: VN. Frequency: annually. Status: In Progress.
- ❑ **Annual policy review** (re-read, increment version). Owner: VN. Frequency: annually. Status: Done (this revision).